Lab 1: Answer Key

INSTRUCTOR USE ONLY

ECON 470 — Microeconomics of Artificial Intelligence

Spring 2026

Table of contents

1 Problem 1: Training on Dinosaurs (30 pts) 2
- 1.1 Part A: The generated names [5 pts] . . . 2
- 1.2 Part B: Interpreting the loss [10 pts] . . . 2
- 1.3 Part C: The context window problem [5 pts] . . . 3
- 1.4 Part D: Expanding the context window [10 pts] . . . 3

2 Problem 2: Backpropagation and the Value Class (36 pts) 5
- 2.1 Part A [10 pts] . . . 5
- 2.2 Part B [16 pts] . . . 5
- 2.3 Part C [10 pts] . . . 6

3 Problem 3: MicroOLS (34 pts) 7
- 3.1 Part A [6 pts] . . . 7
- 3.2 Part B [10 pts] . . . 7
- 3.3 Part C [8 pts] . . . 8
- 3.4 Part D [10 pts] . . . 8

1 Problem 1: Training on Dinosaurs (30 pts)

1.1 Part A: The generated names [5 pts]

Sample output (block_size = 16):

```
sample  1: aculisaurus         sample 11: achanosaurus
sample  2: deranosaurus        sample 12: baloloman
sample  3: peatianatosaurus    sample 13: angangolian
sample  4: anianisaurus        sample 14: gongasaurus
sample  5: tacegisaurus        sample 15: lomananus
sample  6: sancoma             sample 16: busnora
sample  7: pellosaurus         sample 17: pengosaurus
sample  8: periasaurus         sample 18: bamyangosaurus
sample  9: nothangang          sample 19: sananosaurus
sample 10: telodonatoras       sample 20: bingosaurus
```

Key patterns the model picks up:

- The `-saurus` suffix appears in most outputs (as it does in the training data)
- Latin/Greek-sounding syllable patterns (consonant-vowel alternation)
- Common prefixes like `an-`, `para-`, etc.
- Names that are clearly not real dinosaurs but *sound* plausible

Grading: Full credit for copying all 20 names and identifying at least 2 patterns. Accept any reasonable assessment of plausibility.

1.2 Part B: Interpreting the loss [10 pts]

Q2 [5 pts]:

- Initial loss (step 1): ≈ 3.36
- Final loss (step 1000): ≈ 3.01

Converting to probabilities:

- Initial: $P = e^{-3.36} \approx 0.035$ (3.5%). The model is essentially guessing randomly among the 27 possible tokens (26 letters + BOS). This is near the theoretical random baseline of $1/27 \approx 3.7\%$.
- Final: $P = e^{-3.01} \approx 0.049$ (4.9%). After training, the model's average probability of picking the correct next character is about 5%.

Q3 [5 pts]:

The dinosaur final loss (≈ 3.01) is **higher** than the baby names final loss (≈ 2.37). This is because:

- The dinosaur dataset has only 1,835 names vs. 32,033 baby names — less training data

- Dinosaur names are longer on average and use more varied character patterns (Latin/Greek roots from many languages)
- The model has the same capacity (4,192 parameters) but less data to learn from and more complex patterns to capture

Grading: Full credit for correct loss values, correct conversions, and a reasonable explanation mentioning dataset size or pattern complexity.

1.3 Part C: The context window problem [5 pts]

Q4 [5 pts]:

Running the provided code gives:

```
Longest: micropachycephalosaurus (23 chars)
Longer than 16: 45
```

The longest dinosaur name is **micropachycephalosaurus** at **23 characters**. There are **45** dinosaur names longer than 16 characters.

This means the model cannot see the full name during training for those 45 names. When processing character 17 and beyond, the model's positional embeddings are out of range — it was only trained with positions 0–15. For these names, the model is effectively "blind" to the beginning of the name when predicting the ending.

Grading: Full credit for correct numbers (23 chars, 45 names) and explaining the implication for training. Accept minor variations if students count slightly differently.

1.4 Part D: Expanding the context window [10 pts]

Sample output (block_size = 24):

```
sample  1: bengosaurus          sample 11: conganidosaurus
sample  2: tanananananops        sample 12: ptoloman
sample  3: danganisaurus        sample 13: alalantan
sample  4: mamanosaurus         sample 14: tenoceisaurus
sample  5: gastosaurus          sample 15: ileninosaurus
sample  6: dorapasaurus         sample 16: cralonatosaurus
sample  7: ananosaurus          sample 17: araura
sample  8: niangosaurus         sample 18: lacosaurus
sample  9: phatosaurus          sample 19: partoverosaurus
sample 10: pharanosaurus        sample 20: anangora
```

Q5 [5 pts]:

The names are broadly similar in style. Some of the block_size=24 names are longer (e.g., `cralonatosaurus`, `partoverosaurus`), which makes sense: the model can now generate up to 24 characters without hitting the context limit. Students may or may not notice a clear quality difference.

Q6 [5 pts]:

- Initial loss (step 1): ≈ 3.48
- Final loss (step 1000): ≈ 3.13

The initial loss is slightly higher because there are more positional embeddings to initialize (24 vs 16), adding more random parameters. The final loss is similar to or slightly higher than the block_size=16 run. The larger block_size increased the number of parameters (from 4,192 to 4,320) but with only 1,835 training names, the model may not have enough data to take advantage of the extra capacity.

The relationship between architectural capacity and prediction quality is not "more is always better." Capacity only helps when there is enough data to exploit it and when the task requires it. This connects to the economic idea that investing in a more capable system has diminishing returns if the task or data don't justify it.

Grading: Accept reasonable observations about name quality. For loss analysis, credit correct numbers and any reasonable interpretation of the capacity-data tradeoff.

2 Problem 2: Backpropagation and the Value Class (36 pts)

2.1 Part A [10 pts]

Q7 [5 pts]:

The `Value` class wraps a number so that every arithmetic operation (addition, multiplication, exponentiation, etc.) records how the result was computed. It stores *children* (the inputs to each operation) and *local gradients* (the derivative of the output with respect to each input). It needs these to later trace back through the computation and calculate how changing any input would change the final result.

Grading: Full credit for conveying (1) it records computation history, and (2) children + local gradients are needed for backward traversal. Accept informal language.

Q8 [5 pts]:

`L.backward()` computes the derivative of L with respect to every `Value` that contributed to computing L. It starts by setting $\frac{dL}{dL} = 1$, then walks backward through the computation graph in reverse topological order. At each node, it multiplies the local gradient by the incoming gradient and adds it to each child's `.grad` attribute. This accumulation handles cases where a value contributes to the output through multiple paths.

Grading: Full credit for: (1) starts at the output with grad=1, (2) moves backward through the graph, (3) multiplies and accumulates gradients. Exact wording doesn't matter.

2.2 Part B [16 pts]

Q9 [3 pts]:

$$Y = \left(0.5 \cdot 4^{0.5} + 0.5 \cdot 9^{0.5}\right)^{1/0.5} = \left(0.5 \cdot 2 + 0.5 \cdot 3\right)^2 = (1 + 1.5)^2 = 2.5^2 = 6.25$$

Q10 [10 pts]:

Let $Z = \alpha L^\rho + (1-\alpha)K^\rho$ so that $Y = Z^{1/\rho}$.

By the chain rule:

$$\frac{\partial Y}{\partial L} = \frac{1}{\rho} Z^{1/\rho - 1} \cdot \alpha \rho L^{\rho - 1} = \alpha L^{\rho - 1} Z^{1/\rho - 1}$$

Plugging in $\alpha = 0.5$, $\rho = 0.5$, $L = 4$, $Z = 2.5$:

$$MP_L = 0.5 \cdot 4^{-0.5} \cdot 2.5^1 = 0.5 \cdot 0.5 \cdot 2.5 = 0.625$$

Similarly:

$$MP_K = (1-\alpha) K^{\rho - 1} Z^{1/\rho - 1} = 0.5 \cdot 9^{-0.5} \cdot 2.5 = 0.5 \cdot \frac{1}{3} \cdot 2.5 = \frac{5}{12} \approx 0.4167$$

Grading: Full credit for correct final answers with work shown. Partial credit (5/10) for correct setup but arithmetic errors. Accept alternative chain rule formulations that reach the same answer.

Q11 [3 pts]:

Running the code produces:

```
Y    = 6.25
MP_L = 0.625
MP_K = 0.41666666666666663
```

These match the hand calculations: $Y = 6.25$, $MP_L = 0.625$, $MP_K = 5/12 \approx 0.4167$.

Grading: Full credit for running the code and confirming the match. Deduct 1 point if they don't show the output.

2.3 Part C [10 pts]

Q12 [10 pts]:

Every parameter is a `Value` object because the model needs to compute gradients automatically. When `loss.backward()` is called, the chain rule propagates through the entire computation — from the loss, back through the softmax, through the attention mechanism, through the embeddings — all the way to every weight in the model. This gives the training loop a gradient for each of the 4,192 parameters, telling it *which direction* and *how much* to adjust each weight to reduce the loss. The optimizer (Adam) then uses these gradients to update the parameters, making the model's predictions slightly better on the next step.

Grading: Full credit for explaining: (1) `backward()` computes gradients for all parameters, (2) gradients indicate how to adjust parameters to reduce loss, (3) the optimizer uses them. Accept any clear formulation.

3 Problem 3: MicroOLS (34 pts)

3.1 Part A [6 pts]

Q13 [3 pts] — Three things the same:

1. The `Value` class (identical code)
2. The Adam optimizer (same hyperparameters, same update rule)
3. The structure of the training loop: forward pass → compute loss → `loss.backward()` → optimizer step → zero gradients

Q14 [3 pts] — Three things different:

1. The model: a linear equation ($\hat{y} = a \cdot \ln P + b_{mm} \cdot MM + b_{tr} \cdot TR + b_0$) vs. a full transformer (embeddings, attention, MLP)
2. The loss function: mean squared error vs. cross-entropy
3. Inference: sampling from a normal distribution $N(\hat{y}, s_e^2)$ vs. sampling from a categorical distribution over tokens

Grading: Accept any three reasonable items in each category.

3.2 Part B [10 pts]

Q15 [5 pts]:

```
a  = -3.1207  (demand elasticity)
b_mm = 0.8523  (Minute Maid dummy)
b_tr = 1.4959  (Tropicana dummy)
b0 = 10.8151  (intercept)
```

Demand is **elastic** ($|a| = 3.12 > 1$): a 1% increase in price leads to a ~3.1% decrease in quantity demanded. The negative sign confirms the law of demand.

The brand coefficients are both positive, meaning both Minute Maid and Tropicana sell more than Dominick's (the store brand) at the same price. Tropicana's coefficient (1.50) is larger than Minute Maid's (0.85), indicating Tropicana commands higher sales, holding price constant — consistent with stronger brand recognition.

Grading: Full credit for correct parameter values (accept small variations due to randomness), correct elastic/inelastic classification, and reasonable interpretation of brand dummies.

Q16 [5 pts]:

The closed-form OLS gives $a = -3.1387$, $b_{mm} = 0.8702$, $b_{tr} = 1.5299$, $b_0 = 10.8288$.

Gradient descent gives $a = -3.1207$, $b_{mm} = 0.8523$, $b_{tr} = 1.4959$, $b_0 = 10.8151$.

These are very close — within 1–2% of the OLS values. The gradient descent algorithm recovered essentially the same answer as the closed-form formula, despite using a completely different approach (iterative optimization vs. matrix algebra). This demonstrates that gradient descent +

backpropagation is a *general-purpose* optimization engine: it can solve OLS (which has a known formula) just as well as train a GPT (which has no closed-form solution). The same substrate powers both.

Grading: Full credit for noting the values are close and drawing the connection that gradient descent is general-purpose.

3.3 Part C [8 pts]

Q17 [5 pts]:

Sample output at $P = \$3.00$, Tropicana:

```
Q ranges from ~2,300 to ~28,200 units
```

The predicted $\ln(Q) = 8.88$, so $e^{8.88} \approx 7{,}200$ units is the point prediction. But the 20 draws range from roughly 2,000 to 28,000 units — a factor of ~14x between smallest and largest. This reflects the residual standard error ($s_e \approx 0.79$), which is substantial: in log space, $\pm 2 \times 0.79 \approx \pm 1.58$, and $e^{1.58} \approx 4.9$, so a $\pm 2\sigma$ band in log-sales corresponds to nearly a 5x multiplier in units.

Grading: Full credit for reporting the range and noting it is wide. Accept any reasonable reaction to the variation.

Q18 [3 pts]:

Analogous: Both are sampling from a conditional distribution. MicroGPT samples the next character given previous characters; MicroOLS samples sales given price and brand. Both produce a *distribution* of outputs, not a single deterministic answer.

Different: MicroGPT samples from a *categorical* distribution (discrete choice over 27 tokens), while MicroOLS samples from a *normal* distribution (continuous). MicroGPT's distribution is shaped by the entire transformer; MicroOLS's is a normal centered on a linear prediction. MicroGPT's conditional distribution is the model's core output; MicroOLS's noise is added *after* the model, from the residual distribution.

Grading: Full credit for identifying both an analogy and a difference. Accept informal language.

3.4 Part D [10 pts]

Q19 [10 pts]:

A strong answer should touch on these points:

- "Prediction" in MicroGPT means predicting the next character in a sequence. "Prediction" in MicroOLS means predicting sales given price and brand. Both are fundamentally the same operation: the model learns a mapping from inputs to a probability distribution over outputs.
- The *substrate* is identical: `Value` objects track computations, `backward()` computes gradients, and Adam adjusts parameters to minimize a loss function. Whether that loss is cross-entropy (GPT) or sum of squared errors (OLS) doesn't change the machinery.

- *Judgment* is everything the optimization engine cannot provide: the choice of what to predict (characters vs. sales), the choice of loss function (cross-entropy vs. MSE), the choice of model architecture (transformer vs. linear), and what to *do* with the prediction (generate names vs. set prices). These are human decisions that the algorithm takes as given. Gans's point is that AI makes prediction cheap, which makes judgment more valuable — because every cheap prediction needs someone to decide what it's for.

Grading: Full credit for connecting prediction and judgment to both systems and referencing Gans. Partial credit (5/10) for a reasonable but incomplete answer. This is the most open-ended question; grade generously for thoughtful engagement.